Summary Of "Agents in the Physical World"

Donna Pham (phdonn), Toby Buckfire (tqbuck), Dhruv Rawat (dhruvr), Hongkwon Kim (hongkwon)

Section 1: Research direction of the field

Research problem and significance: A robot that takes instructions from a language model has to work with whatever happens in front of it. A grasp can fail if the object shifts. A drone can finish a maneuver before its next command is ready. The first problem is about learning from what happened on the robot. The second is about getting the next command to the robot on time. [1, 2]

State of prior work: Earlier work laid the groundwork for both problems. Code as Policies showed that a language model could write control code that responds to sensor feedback during a task. That work showed how feedback could guide a single execution. ASPIRE takes that idea further by using the outcome of one task to improve code for another. On the serving side, vLLM used memory more efficiently so one server could handle more requests at once. For robots sharing that server, there was still a question of timing. The next command only needs to be ready when the robot can use it. [1, 2, 3, 4]

Contributions of the lecture papers: ASPIRE builds on code generation by using execution traces to see where a program failed. It tests a repair before saving it as a reusable skill. TimelyLLM tackles the timing problem. It generates a plan in pieces so the server can handle other requests while a robot carries out its current action. ASPIRE uses execution feedback to improve future programs. TimelyLLM uses the time an action takes to decide when to generate the next one. [1, 2]

Follow Up Work: Both teams released their implementations on GitHub. In October 2026, the ASPIRE authors revised their paper to add results with Claude Opus 5.5 and an experiment in which a physical robot got better at solving a Rubik's Cube through practice. TimelyLLM first appeared as a preprint in December 2024. Its 2026 MobiSys version evaluates workloads from drones, robot arms, and quadrupeds. As of November 2026, we found no independent work that directly extends either system.

Section 2: Lecture papers

ASPIRE

ASPIRE focuses on improving the reliability of robotic task execution. Traditional Code as Policy approaches allow LLMs to generate executable robot programs, but they often provide limited feedback when execution fails and do not retain successful debugging experiences. To address these limitations, it introduces a robot execution engine that records finegrained execution traces, a skill library that stores validated repairs for future reuse, and a search mechanism that explores multiple candidate programs. The presentation emphesised that accumulating reusable skills allows agents to improve across tasks rather than repeatedly solving the same problems. The skill library stores code snippets, failure patterns, conditions for applying repairs, and verified strategies allowing agents to reuse previous solutions in different situations. Additionally, evolutionary search generates and evaluates multiple candidate programs instead of relying on a single debugging approach, while a coordinator-actor system manages parallel exploration and shared skill updates.

Experimental results demonstrated substantial improvements over conventional Code as Policy methods. On the LIVERO-Pro benchmark, ASPIRE incresed the overall task success rate from 18% to 72%, with particuarly significant improvements in object manipulation tasks. The system also demonstrated successful transfer of skills learned in simulation to physical robots. For example, in a soda-can manipulation task, previously discovered skills reduced the required LLM token usage by approximately 9.4 times while improving the success rate from 13 out of 20 trials to 19 out of 20. However, the presentation also discussed several limitations. Debugging and evolutionary search require numerous LLM calls and simulation rollouts, resulting in considerable computational costs. Additionally, the validation process does not always select the best performing candidate, and the effectivenetss of the system remains dependent on the underlying LLM. These observations suggest that improving planning reliability alone does not eliminate the computational challenges of robotic agents.

Timely LLM

TimelyLLM, focuses on a related but different challenge, which is efficiently scheduling LLM interface for agents interacting with the physical world. The main observation is that LLMs generate instructions much faster than robots can physically execute them. For example, a robot may require several seconds to complete a movement command, while an LLM can generate multiple instructions during the same period. Conventional serving systems, such as vLLM, primarily focus on throughput and inference latency without considering physical execution times. As a result, they may continue generating future instructions even when a robot is already executing its current action, while other robots with more urgent requests wait for GPU resources. TimelyLLM addresses this inefficiency by utilizing the time gap between LLM generation and physical execution.

The system introduces three main techniques: content-aware segmented generation, slack-based scheduling, and latency-guided continuous batching. Instead of generating an entire action sequence at once, TimelyLLM divides the output into meaningful and executable segments. Once a complete instruction is generated, the system can temporarily suspend inference while the robot executes that instruction. The intermediate generation state, including the KV cache, is preserved so that inference can resume efficiently. Meanwhile, the scheduler reallocates computational resources to other agents based on task urgency and remaining execution slack. The presentation particularly emphasized that scheduling priorities should depend not only on the arrival order of requests but also on when each robot needs its next instruction. Latency-guided continuous batching further improves GPU utilization while considering the timing constraints of urgent requests.

Experimental evaluations using robotic agents, including drones, robotic arms, and quadruped robots, demonstrated that TimelyLLM improves time utility by up to 1.52 times and reduces robot waiting time by approximately 84% compared with conventional vLLM serving. The results also showed improvements over traditional scheduling policies such as First-Come-First-Served and Earliest-Deadline-First. However, TimelyLLM relies on estimated action execution times and sufficient slack between consecutive instructions. Its benefits may therefore be limited in tasks requiring immediate feedback from previous actions before generating subsequent instructions.

Section 3: Lecture discussions

ASPIRE

Some tasks get worse as ASPIRE's skill library grows. How do you decide whether a fix should be saved as a skill or kept as a task-specific fix? Proposed skills go through a verifier agent, which independently attempts the source task using only the proposed skill. If the verifier reproduces the success, the coordinator does a final check before adding it to the library. Skills should only be admitted if they generalize, not if they only fixed a single task instance. These declines came from early experiments run before the verifier agent was added.

What happens if the skill library grows to thousands of skills? Is that feasible, and do skills interfere with each other? It isn't very practical with the current stack. It would likely need database-style retrieval tools. It could also use a compaction agent that finds and merges duplicate or similar skills, or a hierarchical library with general skills at the top and specific skills lower down.

How does the system decide whether to use an existing skill or create a new one? What breaks when a redundant skill is added? The paper focuses on how the skill library is accumulated, and the library is kept compact and simple. The system has no explicit mechanism for this problem.

How do you get learned skills to work on a real robot when they initially don't succeed? Simulation skills are used as in-context guidance rather than deployed directly, so the agent still debugs with real-world execution feedback. The system can also keep learning new skills online while operating on the real robot, as in the Rubik's Cube experiment.

TimelyLLM

How sensitive is the scheduler to errors in estimated execution time? How does the system break

down if the estimates are wrong? Execution times come from offline profiling of each atomic robot action, such as move forward or turn. The robot has a fixed set of actions, so estimates generalize to new tasks by composing them. The system uses minimum estimates. Estimation errors reduce the quality of priority ordering but do not affect the correctness of serving.

Can stronger models' skills transfer to weaker models? Do different models end up with the same skill library? Different models learn and apply skills differently. On the "place the book under the shelf" task, Opus 4.6 fails, Opus 5 inserts it into the shelf above the target, and Opus 5.5 knocks the book flat and pushes it under. The models all arrive at different approaches.

TimelyLLM's main benefit comes from reallocating compute. Does it still beat traditional streaming with one GPU? No. Ordinary streaming is better here, and there is little room to schedule when only one request is being served. Segments of a single plan depend on each other, so they can't be reordered, and scheduling reduces to essentially first-come-first-served.

How does the system decide where to segment? The SegmentChecker uses two criteria. The first is structural completeness, in which the tokens must form a complete executable unit. The second is time compliance, where the segment's estimated execution time must exceed a threshold, preventing short segments.

Section 4: Proposal for follow-up work

ASPIRE learns skills offline in simulation. TimelyLLM serves deployed robots by using the time a robot spends moving to handle other robots' requests, but the GPU sits idle under light load. Neither system feeds deployment failures back into a skill library, so failed skills stay broken. So an open question is whether an edge server can keep a fleet responsive and fix failed skills in its idle time.

One promising direction is to run skill repair as a lower-priority workload on the fleet's GPUs. When ASPIRE-style traces show the same failure repeatedly, the scheduler would open a repair job, and failures that waste the most robot time would be fixed first. Each job would fix the skill in a simulated copy of the scene and pause for urgent requests. Ekya (NSDI 2022) made a similar trade between retraining and inference for video analytics. The robot case is harder because repair jobs compete with planning requests for KV cache memory, and edge GPUs hold only smaller models.

A test would run a simulated fleet on one edge server, using TimelyLLM's action timings and adding new objects mid-task in LIBERO-Pro and BEHAVIOR-1K. The baselines are TimelyLLM with online replanning alone, offline ASPIRE with nightly updates, and a fixed GPU share for repair. The idea holds up if the scheduler matches offline ASPIRE's task success with fewer GPUs and no loss in time utility. It fails if repair slows urgent requests or if fixes made in simulation break in real scenes.

Section 5: A reflection on AI usage

Donna: AI helped me see how ASPIRE and TimelyLLM both deal with what happens when a robot carries out a command. The first draft didn't explain clearly how the papers built on earlier work, so I pasted the assignment questions again to get a clearer connection.

Hongkwon: I used AI to better understand the technical details of TimelyLLM, especially how LLM-generated commands interact with robot controllers and how the scheduling mechanism works. I asked follow-up questions to clarify the paper's main contribution and distinguish it from conventional LLM serving systems.

Dhruv: I worked on section 4 of the summary and AI was really useful in doing a really deep survey of the field as well as providing me with some comments on my ideas. I started with using AI to deep dive into some of the assumptions that both papers make and then one by one try to reason about them how the ideas can be generalized. I tried to nudge it to unify ideas from both papers in a way. I also gave the LLM two of my own ideas (which I raised in class during the discussion as well), and AI was able to provide some constructive comments on the same.